

```
In [1]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.preprocessing import StandardScaler, OneHotEncoder, label_binarize
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import StratifiedKFold, GridSearchCV
from sklearn.metrics import accuracy_score, balanced_accuracy_score, cohen_kappa
from sklearn.pipeline import Pipeline
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import BaggingClassifier, RandomForestClassifier
from sklearn.compose import ColumnTransformer
from sklearn.impute import SimpleImputer

from statsmodels.stats.outliers_influence import variance_inflation_factor

import warnings

warnings.filterwarnings('ignore')
```

Modelos predictivos

Regresión logística multinomial con penalización Ridge

```
In [2]: datos_modelos = pd.read_csv("../datos_procesados/datos_con_tipologia_consumo.csv")

datos_train = datos_modelos[datos_modelos["particion"] == "train"].copy()
datos_test = datos_modelos[datos_modelos["particion"] == "test"].copy()

y_train = datos_train["perfil_consumo"]
y_test = datos_test["perfil_consumo"]

X_logit = [
    "tamano_ocde",
    "n_menores",
    "n_mayores",
    "n_ocupados",
    "log_gasto_total_crudo",
    "c_autonoma",
    "tam_municipio",
    "hombre",
    "ocupado",
    "estudios",
    "alquiler",
    "tipo_hogar",
    "edad"
]

X_train_vif = pd.get_dummies(
    datos_train[X_logit],
    drop_first=True,
    dtype=float
```

```
)

X_train_vif = X_train_vif.loc[:, X_train_vif.std() > 0]

X_train_vif = X_train_vif.fillna(X_train_vif.median())

vif_resultados = []

for i, col in enumerate(X_train_vif.columns):
    vif_resultados.append({
        "variable": col,
        "VIF": variance_inflation_factor(X_train_vif.values, i)
    })

tabla_vif = (
    pd.DataFrame(vif_resultados)
    .sort_values("VIF", ascending=False)
)

display(tabla_vif)
```

	variable	VIF
0	tamano_ocde	1264.157815
2	n_mayores	716.548768
4	log_gasto_total_crudo	106.057842
1	n_menores	72.103096
37	edad_46-65	20.219633
38	edad_Más de 65	14.883423
6	ocupado	9.899095
36	edad_31-45	9.421406
3	n_ocupados	8.999922
32	tipo_hogar_Pareja con hijos	6.854840
27	tam_municipio_Muy Grande	4.236159
33	tipo_hogar_Pareja sin hijos	4.232772
28	tam_municipio_Pequeño	4.049391
5	hombre	3.672454
30	estudios_Medio	3.343618
34	tipo_hogar_Persona sola <65	3.086787
35	tipo_hogar_Persona sola ≥65	2.817164
26	tam_municipio_Mediano	2.484715
31	tipo_hogar_Otros hogares	2.253615
25	c_autonoma_País Vasco	1.988643
29	estudios_Bajo	1.918505
21	c_autonoma_Madrid	1.884819
16	c_autonoma_Comunidad Valenciana	1.727537
14	c_autonoma_Cataluña	1.709748
18	c_autonoma_Galicia	1.621542
12	c_autonoma_Castilla y León	1.582826
13	c_autonoma_Castilla-La Mancha	1.476944
17	c_autonoma_Extremadura	1.469869
10	c_autonoma_Canarias	1.440062
23	c_autonoma_Murcia	1.426071
9	c_autonoma_Asturias	1.415226
8	c_autonoma_Aragón	1.414438
11	c_autonoma_Cantabria	1.381191

	variable	VIF
24	c_autonoma_Navarra	1.327512
20	c_autonoma_La Rioja	1.305014
7	alquiler	1.296515
19	c_autonoma_Islas Baleares	1.187572
15	c_autonoma_Ceuta	1.092725
22	c_autonoma_Melilla	1.085181

```
In [3]: X_train_final = pd.get_dummies(
        datos_train[X_logit],
        drop_first=True,
        dtype=float
    )

    X_test_final = pd.get_dummies(
        datos_test[X_logit],
        drop_first=True,
        dtype=float
    )

    X_train_final, X_test_final = X_train_final.align(
        X_test_final,
        join="left",
        axis=1,
        fill_value=0
    )

    y_train_final = datos_train["perfil_consumo"]
    y_test_final = datos_test["perfil_consumo"]

    print("X_train_final:", X_train_final.shape)
    print("X_test_final:", X_test_final.shape)
    print("y_train_final:", y_train_final.shape)
    print("y_test_final:", y_test_final.shape)

    dist_train = y_train_final.value_counts(normalize=True).sort_index()
    dist_test = y_test_final.value_counts(normalize=True).sort_index()
    comparativa_estratificacion = pd.DataFrame({
        "Proporción Train (%)": (dist_train * 100).round(2),
        "Proporción Test (%)": (dist_test * 100).round(2)
    })
    display(comparativa_estratificacion)
```

X_train_final: (14493, 39)

X_test_final: (6212, 39)

y_train_final: (14493,)

y_test_final: (6212,)

	Proporción Train (%)	Proporción Test (%)
perfil_consumo		
Alcohol y tabaco	5.24	5.20
Bienes, servicios e imagen personal	9.74	9.71
Consumo básico doméstico	20.91	19.74
Educación	3.35	3.91
Equipamiento del hogar	3.58	3.22
Movilidad intensiva	10.49	10.54
Ocio y restauración	20.91	19.83
Salud	6.20	6.21
Vivienda	19.59	21.64

```
In [4]: modelo_logit_ridge = Pipeline(steps=[
    ("imputer", SimpleImputer(strategy="median")),
    ("scaler", StandardScaler()),
    ("logit", LogisticRegression(
        penalty="l2",
        solver="lbfgs",
        class_weight="balanced",
        max_iter=5000,
        random_state=111104
    ))
])

cv = StratifiedKFold(
    n_splits=5,
    shuffle=True,
    random_state=111104
)

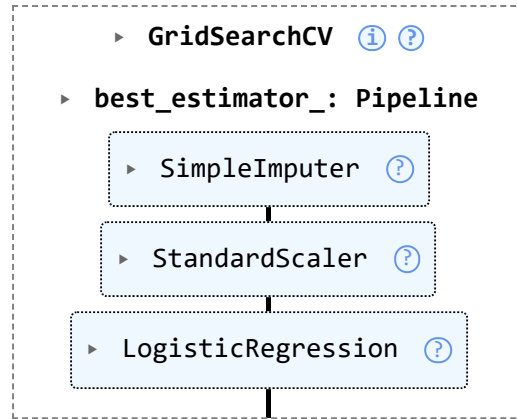
lambdas_ridge = np.arange(0.01, 10, 0.01)

param_grid_ridge = {
    "logit__C": list(1 / lambdas_ridge)
}

grid_logit_ridge = GridSearchCV(
    estimator=modelo_logit_ridge,
    param_grid=param_grid_ridge,
    scoring={
        "accuracy": "accuracy",
        "balanced_accuracy": "balanced_accuracy",
        "f1_macro": "f1_macro"
    },
    refit="balanced_accuracy",
    cv=cv,
    n_jobs=-1,
    return_train_score=True
)
```

```
grid_logit_ridge.fit(X_train_final, y_train_final)
```

Out[4]:



In [5]:

```
resultados_cv_logit = pd.DataFrame(grid_logit_ridge.cv_results_)
resultados_cv_logit["lambda"] = 1 / resultados_cv_logit["param_logit_C"].astype(float)

mejor_media = resultados_cv_logit["mean_test_balanced_accuracy"].max()

mejores_logit = resultados_cv_logit[
    resultados_cv_logit["mean_test_balanced_accuracy"] >= mejor_media - resultados_cv_logit["std_test_balanced_accuracy"]
].copy()

mejores_logit = mejores_logit.sort_values(
    ["mean_test_balanced_accuracy", "std_test_balanced_accuracy"],
    ascending=[False, True]
)

display(mejores_logit[
    [
        "lambda",
        "param_logit_C",
        "mean_test_balanced_accuracy",
        "std_test_balanced_accuracy",
        "mean_test_f1_macro",
        "std_test_f1_macro",
        "mean_train_balanced_accuracy"
    ]
].head(5).round(4))
```

	lambda	param_logit_C	mean_test_balanced_accuracy	std_test_balanced_accuracy	m
247	2.48	0.4032	0.2772	0.0065	
519	5.20	0.1923	0.2772	0.0066	
235	2.36	0.4237	0.2771	0.0063	
234	2.35	0.4255	0.2771	0.0064	
520	5.21	0.1919	0.2771	0.0065	

In [6]:

```
resultados_cv_logit = pd.DataFrame(grid_logit_ridge.cv_results_)

mejores_logit = resultados_cv_logit.sort_values(by="mean_test_balanced_accuracy")

mejores_logit["lambda_val"] = mejores_logit["params"].apply(lambda x: 1.0 / float(x["param_logit_C"]))
mejores_logit["etiqueta_eje_y"] = mejores_logit["lambda_val"].apply(lambda x: f"lambda_{x}")
```

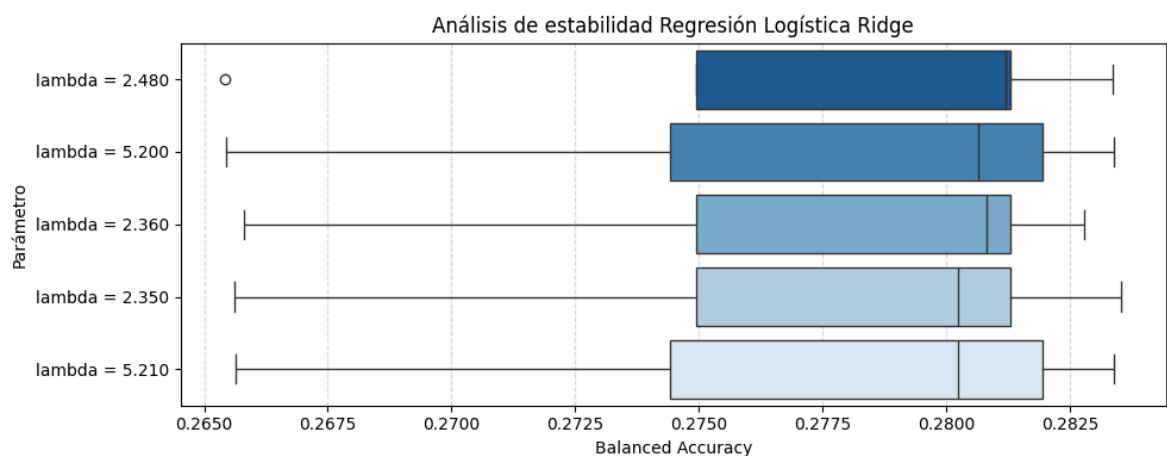
```

columnas_splits_logit = [f"split{i}_test_balanced_accuracy" for i in range(5)]

df_boxplot_logit = mejores_logit.melt(
    id_vars=["etiqueta_eje_y"],
    value_vars=columnas_splits_logit,
    var_name="Fold",
    value_name="Balanced Accuracy"
)

plt.figure(figsize=(10, 4))
sns.boxplot(data=df_boxplot_logit, x="Balanced Accuracy", y="etiqueta_eje_y", palette="magma")
plt.title("Análisis de estabilidad Regresión Logística Ridge")
plt.xlabel("Balanced Accuracy")
plt.ylabel("Parámetro")
plt.grid(axis='x', linestyle='--', alpha=0.5)
plt.tight_layout()
plt.show()

```



```

In [7]: lambda_elegida = 2.36

modelo_logit_ridge_final = Pipeline(steps=[
    ("imputer", SimpleImputer(strategy="median")),
    ("scaler", StandardScaler()),
    ("logit", LogisticRegression(
        penalty="l2",
        C= 1 / lambda_elegida,
        solver="lbfgs",
        class_weight="balanced",
        max_iter=5000,
        random_state=111104
    ))
])

modelo_logit_ridge_final.fit(X_train_final, y_train_final)

def evaluar_clasificador(nombre_modelo, muestra, modelo, X, y):
    y_pred = modelo.predict(X)

    return {
        "modelo": nombre_modelo,
        "muestra": muestra,
        "accuracy": accuracy_score(y, y_pred),
        "balanced_accuracy": balanced_accuracy_score(y, y_pred),
        "f1_macro": f1_score(y, y_pred, average="macro"),
    }

```

```

        "kappa": cohen_kappa_score(y, y_pred)
    }

metricas_logit_ridge = pd.DataFrame([
    evaluar_clasificador(
        "Logística multinomial Ridge",
        "train",
        modelo_logit_ridge_final,
        X_train_final,
        y_train_final
    ),
    evaluar_clasificador(
        "Logística multinomial Ridge",
        "test",
        modelo_logit_ridge_final,
        X_test_final,
        y_test_final
    )
])

display(metricas_logit_ridge.round(4))

```

	modelo	muestra	accuracy	balanced_accuracy	f1_macro	kappa
0	Logística multinomial Ridge	train	0.2674	0.2915	0.2290	0.1772
1	Logística multinomial Ridge	test	0.2601	0.2579	0.2128	0.1649

```

In [8]: y_pred_test_ridge = modelo_logit_ridge_final.predict(X_test_final)

print(classification_report(
    y_test_final,
    y_pred_test_ridge
))

```

	precision	recall	f1-score	support
Alcohol y tabaco	0.10	0.24	0.14	323
Bienes, servicios e imagen personal	0.18	0.09	0.12	603
Consumo básico doméstico	0.38	0.16	0.22	1226
Educación	0.12	0.54	0.19	243
Equipamiento del hogar	0.07	0.20	0.10	200
Movilidad intensiva	0.27	0.29	0.28	655
Ocio y restauración	0.41	0.14	0.21	1232
Salud	0.11	0.15	0.13	386
Vivienda	0.53	0.53	0.53	1344
accuracy			0.26	6212
macro avg	0.24	0.26	0.21	6212
weighted avg	0.33	0.26	0.27	6212

```

In [9]: labels_ridge = modelo_logit_ridge_final.named_steps["logit"].classes_

matriz_logit_ridge_norm = confusion_matrix(
    y_test_final,
    y_pred_test_ridge,
    labels=labels_ridge,
    normalize="true"
)

```



```
plt.figure(figsize=(10, 8))
sns.heatmap(
    matriz_logit_ridge_norm,
    annot=True,
    fmt=".2f",
    cmap="Blues",
    xticklabels=labels_ridge,
    yticklabels=labels_ridge,
)
plt.title("Matriz de confusión normalizada - Logística multinomial Ridge")
plt.xlabel("Clase predicha")
plt.ylabel("Clase real")
plt.tight_layout()
plt.show()
```



Árbol de clasificación

```
In [10]: suple_numericas = [
    "tamano_ocde", "n_menores", "n_mayores", "n_ocupados",
    "log_gasto_total_crudo"
]

suple_categoricas = [
    "c_autonoma", "tam_municipio", "hombre",
    "ocupado", "estudios", "alquiler",
    "tipo_hogar", "edad"
]
```

```

X_train_arbol = datos_train[suple_numericas + suple_categoricas].copy()
y_train_arbol = datos_train["perfil_consumo"].copy()

X_test_arbol = datos_test[suple_numericas + suple_categoricas].copy()
y_test_arbol = datos_test["perfil_consumo"].copy()

preprocess_arbol = ColumnTransformer(
    transformers=[
        ("num", SimpleImputer(strategy="median"), suple_numericas),
        ("cat", Pipeline(steps=[
            ("imputer", SimpleImputer(strategy="most_frequent")),
            ("onehot", OneHotEncoder(handle_unknown="ignore"))
        ]), suple_categoricas)
    ],
    remainder="drop"
)

```

In [11]: `X_train_arbol_proc = preprocess_arbol.fit_transform(X_train_arbol)`

```

arbol_para_poda = DecisionTreeClassifier(
    random_state=111104,
    class_weight="balanced",
    criterion="gini",
    min_samples_leaf=25
)

arbol_para_poda.fit(X_train_arbol_proc, y_train_arbol)

path = arbol_para_poda.cost_complexity_pruning_path(
    X_train_arbol_proc,
    y_train_arbol
)

ccp_alphas = path.ccp_alphas[:-1]
ccp_alphas_pos = ccp_alphas[ccp_alphas > 0]

if len(ccp_alphas_pos) > 0:
    ccp_alpha_grid = np.unique(
        np.quantile(ccp_alphas_pos, q=np.linspace(0, 0.95, 8))
    )
    ccp_alpha_grid = np.insert(ccp_alpha_grid, 0, 0.0)
else:
    ccp_alpha_grid = np.array([0.0])

print("Valores de ccp_alpha considerados:")
print(ccp_alpha_grid.round(6))

```

Valores de ccp_alpha considerados:

```

[0.00e+00 3.00e-06 7.40e-05 1.05e-04 1.34e-04 1.64e-04 2.16e-04 2.93e-04
 6.11e-04]

```

In [12]: `cv = StratifiedKFold(`

```

    n_splits=5,
    shuffle=True,
    random_state=111104
)

modelo_arbol = Pipeline(steps=[
    ("preprocess", preprocess_arbol),
    ("tree", DecisionTreeClassifier(

```

```

        random_state=111104,
        class_weight="balanced"
    ))
])

n_train_arbol = len(y_train_arbol)

min_samples_leaf_grid = [
    int(np.ceil(0.003 * n_train_arbol)),
    int(np.ceil(0.005 * n_train_arbol)),
    int(np.ceil(0.010 * n_train_arbol))
]

min_samples_split_grid = [
    int(np.ceil(0.010 * n_train_arbol)),
    int(np.ceil(0.020 * n_train_arbol)),
    int(np.ceil(0.040 * n_train_arbol))
]

param_grid_arbol = {
    "tree__criterion": ["gini", "entropy"],
    "tree__max_depth": [3, 4, 5, 6, 8, 10, 12, 14, 16],
    "tree__min_samples_split": min_samples_split_grid,
    "tree__min_samples_leaf": min_samples_leaf_grid,
    "tree__ccp_alpha": [0.0]
}

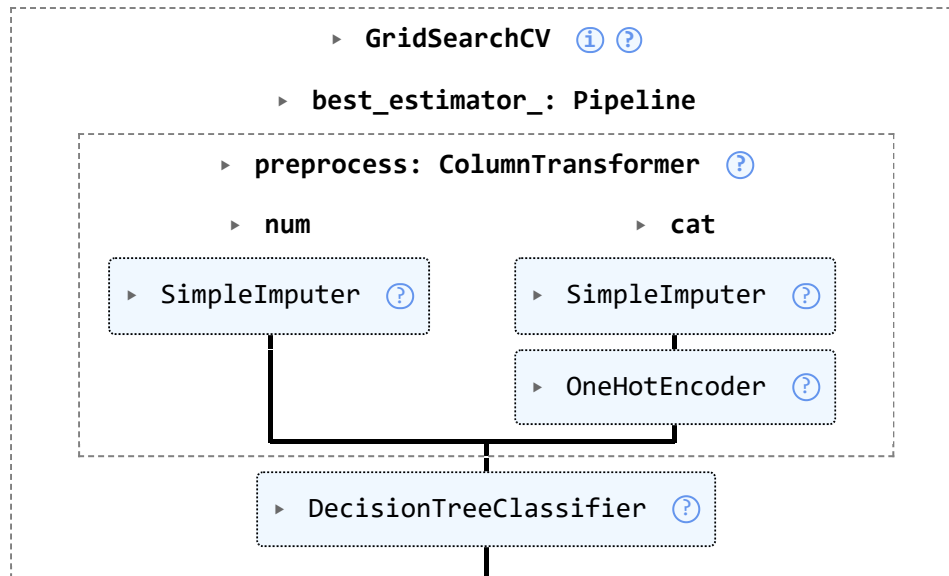
grid_arbol = GridSearchCV(
    estimator=modelo_arbol,
    param_grid=param_grid_arbol,
    scoring={
        "accuracy": "accuracy",
        "balanced_accuracy": "balanced_accuracy",
        "f1_macro": "f1_macro"
    },
    refit="balanced_accuracy",
    cv=cv,
    n_jobs=-1,
    verbose=1,
    return_train_score=True
)

grid_arbol.fit(X_train_arbol, y_train_arbol)

```

Fitting 5 folds for each of 162 candidates, totalling 810 fits

Out[12]:



```
In [13]: resultados_cv_arbol = pd.DataFrame(grid_arbol.cv_results_).copy()

resultados_cv_arbol["criterion"] = resultados_cv_arbol["param_tree__criterion"]
resultados_cv_arbol["max_depth"] = resultados_cv_arbol["param_tree__max_depth"]
resultados_cv_arbol["min_samples_split"] = resultados_cv_arbol["param_tree__min_
resultados_cv_arbol["min_samples_leaf"] = resultados_cv_arbol["param_tree__min_s

cols_arbol = [
    "criterion",
    "max_depth",
    "min_samples_split",
    "min_samples_leaf",
    "mean_test_accuracy",
    "std_test_accuracy",
    "mean_test_balanced_accuracy",
    "std_test_balanced_accuracy",
    "mean_test_f1_macro",
    "std_test_f1_macro",
    "mean_train_balanced_accuracy"
]

resultados_arbol_ordenados = resultados_cv_arbol[cols_arbol].sort_values(
    "mean_test_balanced_accuracy",
    ascending=False
)

display(resultados_arbol_ordenados.head(15).round(4))
```

	criterion	max_depth	min_samples_split	min_samples_leaf	mean_test_accuracy	std_t
130	entropy	10	290	73	0.2237	
157	entropy	16	290	73	0.2238	
139	entropy	12	290	73	0.2238	
148	entropy	14	290	73	0.2238	
121	entropy	8	290	73	0.2237	
49	gini	10	290	73	0.2134	
58	gini	12	290	73	0.2140	
76	gini	16	290	73	0.2140	
67	gini	14	290	73	0.2140	
112	entropy	6	290	73	0.2234	
114	entropy	6	145	145	0.2169	
115	entropy	6	290	145	0.2169	
46	gini	10	290	44	0.2111	
77	gini	16	580	73	0.2169	
50	gini	10	580	73	0.2169	

```
In [14]: top_arbol = resultados_cv_arbol.sort_values(
        by="mean_test_balanced_accuracy",
        ascending=False
    ).head(5).copy()

    top_arbol["etiqueta_eje_y"] = top_arbol.apply(
        lambda r: (
            f"depth={int(r['max_depth'])}, "
            f"leaf={int(r['min_samples_leaf'])}, "
            f"split={int(r['min_samples_split'])}, "
            f"criterion={r['criterion']}"
        ),
        axis=1
    )

    columnas_splits_arbol = [f"split{i}_test_balanced_accuracy" for i in range(5)]

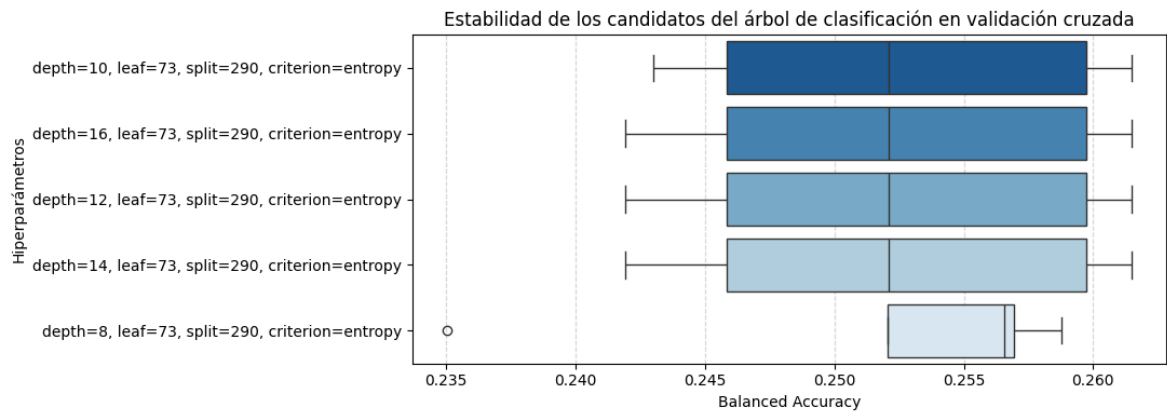
    df_boxplot_arbol = top_arbol.melt(
        id_vars=["etiqueta_eje_y"],
        value_vars=columnas_splits_arbol,
        var_name="Fold",
        value_name="Balanced Accuracy"
    )

    plt.figure(figsize=(11, 4))
    sns.boxplot(
        data=df_boxplot_arbol,
        x="Balanced Accuracy",
        y="etiqueta_eje_y",
```

```

    palette="Blues_r",
    orient="h"
)
plt.title("Estabilidad de los candidatos del árbol de clasificación en validación cruzada")
plt.xlabel("Balanced Accuracy")
plt.ylabel("Hiperparámetros")
plt.grid(axis="x", linestyle="--", alpha=0.5)
plt.tight_layout()
plt.show()

```



```

In [15]: mejor_arbol = Pipeline(steps=[
    ("preprocess", preprocess_arbol),
    ("tree", DecisionTreeClassifier(
        random_state=111104,
        class_weight="balanced",
        criterion="entropy",
        max_depth=10,
        min_samples_split=290,
        min_samples_leaf=73,
        ccp_alpha=0.0
    ))
])

mejor_arbol.fit(X_train_arbol, y_train_arbol)

def evaluar_clasificador(nombre_modelo, muestra, modelo, X, y):
    y_pred = modelo.predict(X)

    return {
        "modelo": nombre_modelo,
        "muestra": muestra,
        "accuracy": accuracy_score(y, y_pred),
        "balanced_accuracy": balanced_accuracy_score(y, y_pred),
        "f1_macro": f1_score(y, y_pred, average="macro"),
        "kappa": cohen_kappa_score(y, y_pred)
    }

metricas_arbol = pd.DataFrame([
    evaluar_clasificador(
        "Árbol de clasificación",
        "train",
        mejor_arbol,
        X_train_arbol,
        y_train_arbol
    ),
    evaluar_clasificador(
        "Árbol de clasificación",

```

```

        "test",
        mejor_arbol,
        X_test_arbol,
        y_test_arbol
    )
])

display(metricas_arbol.round(4))

```

	modelo	muestra	accuracy	balanced_accuracy	f1_macro	kappa
0	Árbol de clasificación	train	0.2412	0.2867	0.2090	0.1555
1	Árbol de clasificación	test	0.2165	0.2379	0.1805	0.1269

```

In [16]: y_pred_arbol = mejor_arbol.predict(X_test_arbol)

print(classification_report(
    y_test_arbol,
    y_pred_arbol
))

labels_arbol = mejor_arbol.named_steps["tree"].classes_

matriz_arbol_norm = confusion_matrix(
    y_test_arbol,
    y_pred_arbol,
    labels=labels_arbol,
    normalize="true"
)

soportes = y_test_arbol.value_counts().to_dict()

labels_con_n = [
    f"{label}"
    for label in labels_arbol
]

plt.figure(figsize=(10, 8))
sns.heatmap(
    matriz_arbol_norm,
    annot=True,
    fmt=".2f",
    cmap="Blues",
    xticklabels=labels_arbol,
    yticklabels=labels_con_n
)
plt.title("Matriz de confusión normalizada - Árbol de clasificación")
plt.xlabel("Clase predicha")
plt.ylabel("Clase real")
plt.tight_layout()
plt.show()

```

	precision	recall	f1-score	support
Alcohol y tabaco	0.07	0.33	0.12	323
Bienes, servicios e imagen personal	0.15	0.04	0.07	603
Consumo básico doméstico	0.35	0.19	0.25	1226
Educación	0.12	0.63	0.20	243
Equipamiento del hogar	0.08	0.18	0.11	200
Movilidad intensiva	0.25	0.24	0.24	655
Ocio y restauración	0.39	0.06	0.11	1232
Salud	0.11	0.08	0.09	386
Vivienda	0.51	0.39	0.44	1344
accuracy			0.22	6212
macro avg	0.22	0.24	0.18	6212
weighted avg	0.31	0.22	0.22	6212



Bagging

```
In [17]: X_train_bagging = datos_train[suple_numericas + suple_categoricas].copy()
y_train_bagging = datos_train["perfil_consumo"].copy()

X_test_bagging = datos_test[suple_numericas + suple_categoricas].copy()
y_test_bagging = datos_test["perfil_consumo"].copy()

preprocess_bagging = ColumnTransformer(
    transformers=[
        ("num", SimpleImputer(strategy="median"), suple_numericas),
        ("cat", Pipeline(steps=[
```



```

        ("imputer", SimpleImputer(strategy="most_frequent")),
        ("onehot", OneHotEncoder(handle_unknown="ignore"))
    ]), suple_categoricas)
],
remainder="drop"
)

```

```

In [18]: cv = StratifiedKFold(
    n_splits=5,
    shuffle=True,
    random_state=111104
)

arbol_base_bagging = DecisionTreeClassifier(
    criterion="entropy",
    class_weight="balanced",
    random_state=111104
)

bagging = BaggingClassifier(
    estimator=arbol_base_bagging,
    bootstrap=True,
    max_features=1.0,
    bootstrap_features=False,
    oob_score=True,
    random_state=111104,
    n_jobs=-1
)

modelo_bagging = Pipeline(steps=[
    ("preprocess", preprocess_bagging),
    ("bagging", bagging)
])

param_grid_bagging = {
    "bagging__n_estimators": [300, 500],
    "bagging__max_samples": [0.8, 1.0],
    "bagging__estimator__max_depth": [None, 10, 16],
    "bagging__estimator__min_samples_leaf": [25, 50, 73],
    "bagging__estimator__min_samples_split": [145, 290],
    "bagging__estimator__criterion": ["entropy"]
}

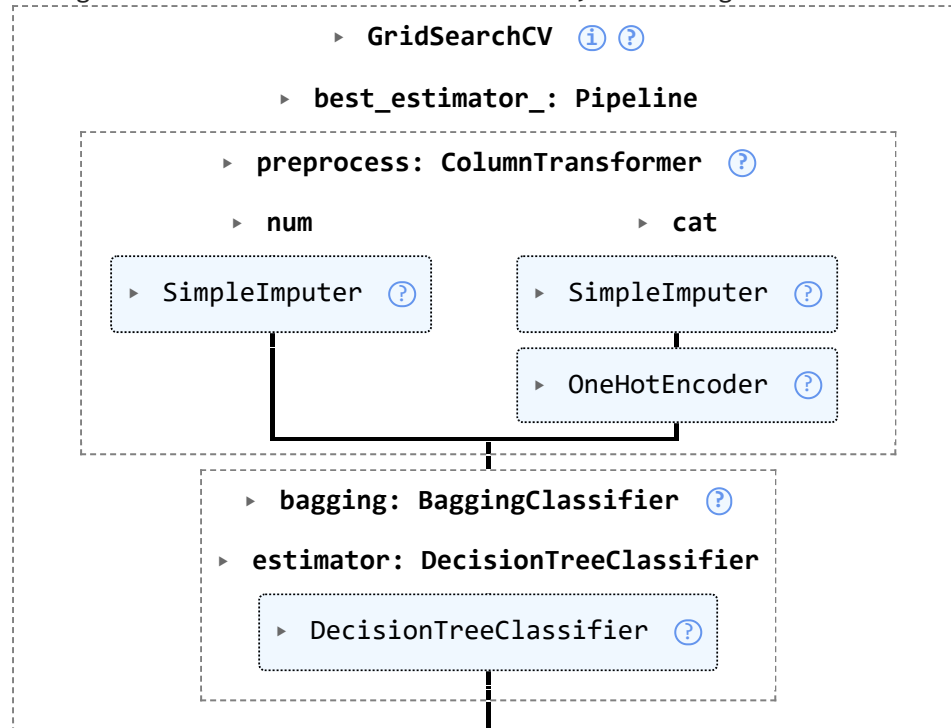
grid_bagging = GridSearchCV(
    estimator=modelo_bagging,
    param_grid=param_grid_bagging,
    scoring={
        "accuracy": "accuracy",
        "balanced_accuracy": "balanced_accuracy",
        "f1_macro": "f1_macro"
    },
    refit="balanced_accuracy",
    cv=cv,
    n_jobs=1,
    verbose=1,
    return_train_score=True
)

```

```
grid_bagging.fit(X_train_bagging, y_train_bagging)
```

Fitting 5 folds for each of 72 candidates, totalling 360 fits

Out[18]:



```
In [19]: resultados_cv_bagging = pd.DataFrame(grid_bagging.cv_results_).copy()

resultados_cv_bagging["n_estimators"] = resultados_cv_bagging["param_bagging_n"]
resultados_cv_bagging["max_samples"] = resultados_cv_bagging["param_bagging_max"]
resultados_cv_bagging["max_depth"] = resultados_cv_bagging["param_bagging_estim"]
resultados_cv_bagging["min_samples_leaf"] = resultados_cv_bagging["param_bagging_min"]
resultados_cv_bagging["min_samples_split"] = resultados_cv_bagging["param_bagging_split"]
resultados_cv_bagging["criterion"] = resultados_cv_bagging["param_bagging_criterion"]

cols_bagging = [
    "n_estimators",
    "max_samples",
    "criterion",
    "max_depth",
    "min_samples_split",
    "min_samples_leaf",
    "mean_test_accuracy",
    "std_test_accuracy",
    "mean_test_balanced_accuracy",
    "std_test_balanced_accuracy",
    "mean_test_f1_macro",
    "std_test_f1_macro",
    "mean_train_balanced_accuracy"
]

resultados_bagging_ordenados = resultados_cv_bagging[cols_bagging].sort_values(
    "mean_test_balanced_accuracy",
    ascending=False
)

display(resultados_bagging_ordenados.head(15).round(4))
```

	n_estimators	max_samples	criterion	max_depth	min_samples_split	min_samples_lea
48	300	0.8	entropy	16	145	2!
9	500	0.8	entropy	None	145	50
57	500	0.8	entropy	16	145	50
0	300	0.8	entropy	None	145	2!
1	500	0.8	entropy	None	145	2!
24	300	0.8	entropy	10	145	2!
49	500	0.8	entropy	16	145	2!
33	500	0.8	entropy	10	145	50
34	300	1.0	entropy	10	145	50
5	500	0.8	entropy	None	290	2!
53	500	0.8	entropy	16	290	2!
52	300	0.8	entropy	16	290	2!
4	300	0.8	entropy	None	290	2!
29	500	0.8	entropy	10	290	2!
28	300	0.8	entropy	10	290	2!

```
In [20]: top_bagging = resultados_cv_bagging.sort_values(
        by="mean_test_balanced_accuracy",
        ascending=False
    ).head(5).copy()

    top_bagging["etiqueta_eje_y"] = top_bagging.apply(
        lambda r: (
            f"B={int(r['n_estimators'])}, "
            f"sample={r['max_samples']:.1f}, "
            f"depth={r['max_depth']}, "
            f"leaf={int(r['min_samples_leaf'])}, "
            f"split={int(r['min_samples_split'])}"
        ),
        axis=1
    )

    columnas_splits_bagging = [f"split{i}_test_balanced_accuracy" for i in range(5)]

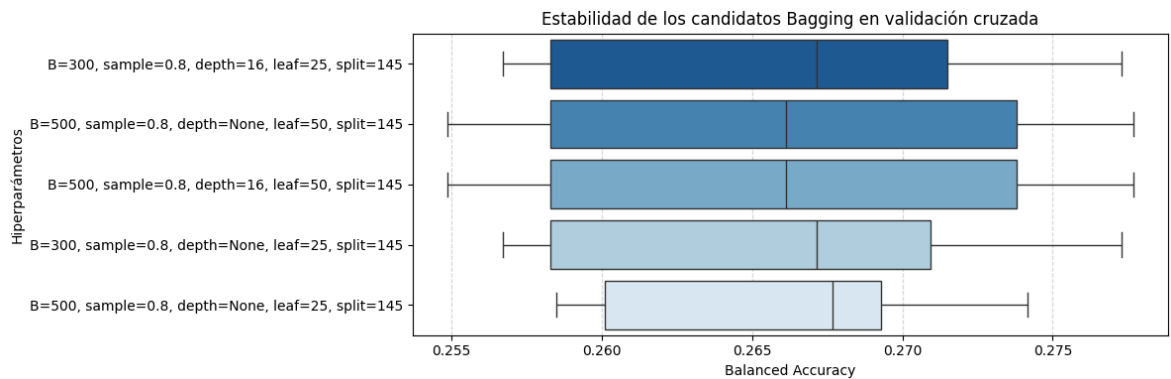
    df_boxplot_bagging = top_bagging.melt(
        id_vars=["etiqueta_eje_y"],
        value_vars=columnas_splits_bagging,
        var_name="Fold",
        value_name="Balanced Accuracy"
    )

    plt.figure(figsize=(12, 4))
    sns.boxplot(
        data=df_boxplot_bagging,
        x="Balanced Accuracy",
```

```

    y="etiqueta_eje_y",
    palette="Blues_r",
    orient="h"
)
plt.title("Estabilidad de los candidatos Bagging en validación cruzada")
plt.xlabel("Balanced Accuracy")
plt.ylabel("Hiperparámetros")
plt.grid(axis="x", linestyle="--", alpha=0.5)
plt.tight_layout()
plt.show()

```



```

In [21]: mejor_bagging = Pipeline(steps=[
    ("preprocess", preprocess_bagging),
    ("bagging", BaggingClassifier(
        estimator=DecisionTreeClassifier(
            criterion="entropy",
            max_depth=10,
            min_samples_split=290,
            min_samples_leaf=25,
            class_weight="balanced",
            random_state=111104
        ),
        n_estimators=300,
        max_samples=0.8,
        bootstrap=True,
        max_features=1.0,
        bootstrap_features=False,
        oob_score=True,
        random_state=111104,
        n_jobs=-1
    ))
])

mejor_bagging.fit(X_train_bagging, y_train_bagging)

def evaluar_clasificador(nombre_modelo, muestra, modelo, X, y):
    y_pred = modelo.predict(X)

    return {
        "modelo": nombre_modelo,
        "muestra": muestra,
        "accuracy": accuracy_score(y, y_pred),
        "balanced_accuracy": balanced_accuracy_score(y, y_pred),
        "f1_macro": f1_score(y, y_pred, average="macro"),
        "kappa": cohen_kappa_score(y, y_pred)
    }

metricas_bagging = pd.DataFrame([

```

```

    evaluar_clasificador(
        "Bagging",
        "train",
        mejor_bagging,
        X_train_bagging,
        y_train_bagging
    ),
    evaluar_clasificador(
        "Bagging",
        "test",
        mejor_bagging,
        X_test_bagging,
        y_test_bagging
    )
])

display(metricas_bagging.round(4))

```

	modelo	muestra	accuracy	balanced_accuracy	f1_macro	kappa
0	Bagging	train	0.2676	0.2971	0.2264	0.1755
1	Bagging	test	0.2413	0.2431	0.1937	0.1428

```

In [22]: y_pred_bagging = mejor_bagging.predict(X_test_bagging)

print(classification_report(
    y_test_bagging,
    y_pred_bagging,
    zero_division=0
))

labels_bagging = mejor_bagging.named_steps["bagging"].classes_

matriz_bagging_norm = confusion_matrix(
    y_test_bagging,
    y_pred_bagging,
    labels=labels_bagging,
    normalize="true"
)

soportes = y_test_bagging.value_counts().to_dict()

labels_con_n = [
    f"{label} (n={soportes[label]})"
    for label in labels_bagging
]

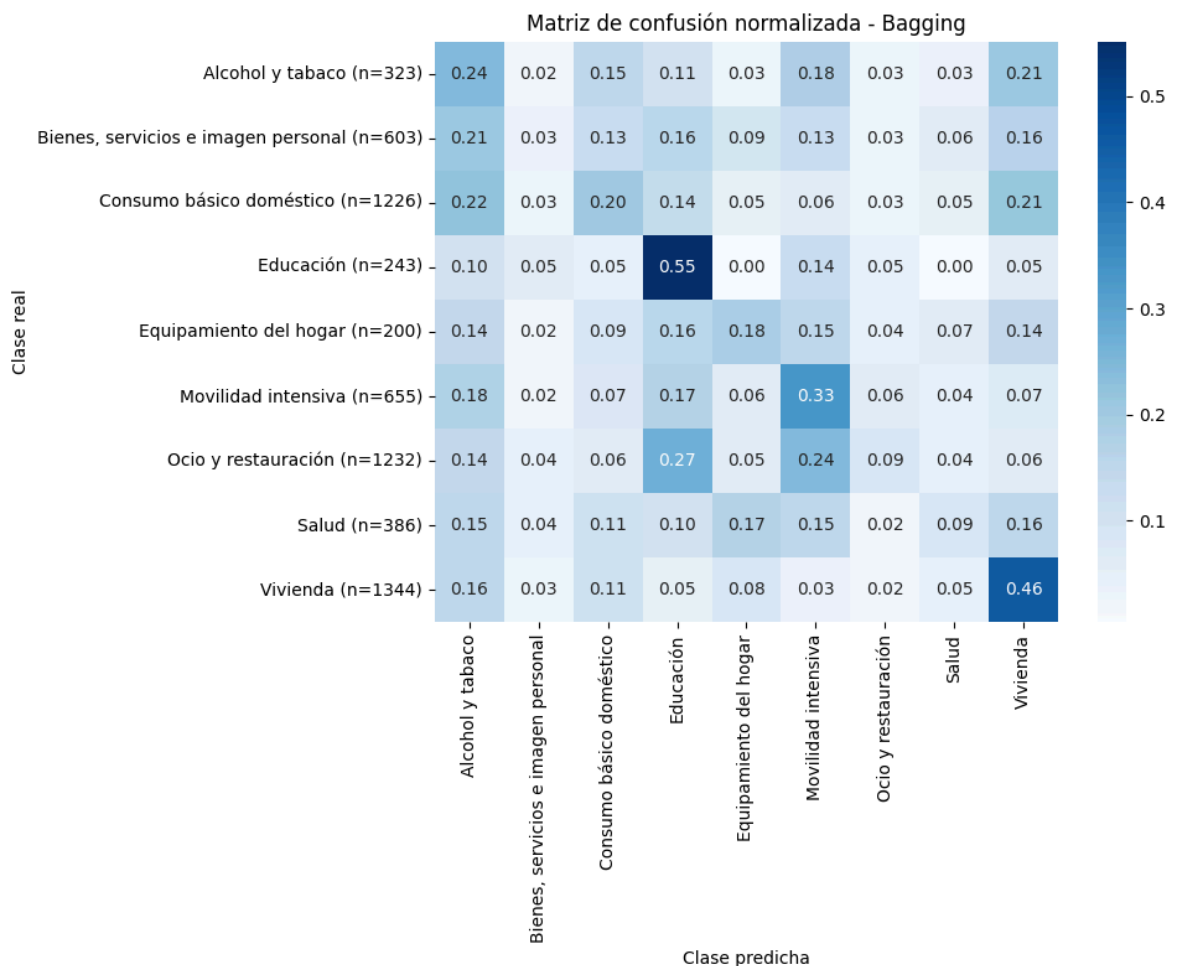
plt.figure(figsize=(10, 8))
sns.heatmap(
    matriz_bagging_norm,
    annot=True,
    fmt=".2f",
    cmap="Blues",
    xticklabels=labels_bagging,
    yticklabels=labels_con_n
)

plt.title("Matriz de confusión normalizada - Bagging")

```

```
plt.xlabel("Clase predicha")
plt.ylabel("Clase real")
plt.tight_layout()
plt.show()
```

	precision	recall	f1-score	support
Alcohol y tabaco	0.07	0.24	0.11	323
Bienes, servicios e imagen personal	0.11	0.03	0.05	603
Consumo básico doméstico	0.34	0.20	0.25	1226
Educación	0.13	0.55	0.21	243
Equipamiento del hogar	0.08	0.18	0.11	200
Movilidad intensiva	0.24	0.33	0.28	655
Ocio y restauración	0.39	0.09	0.14	1232
Salud	0.11	0.09	0.10	386
Vivienda	0.49	0.46	0.48	1344
accuracy			0.24	6212
macro avg	0.22	0.24	0.19	6212
weighted avg	0.31	0.24	0.24	6212



Random Forest

```
In [23]: X_train_rf = datos_train[suple_numericas + suple_categoricas].copy()
y_train_rf = datos_train["perfil_consumo"].copy()

X_test_rf = datos_test[suple_numericas + suple_categoricas].copy()
y_test_rf = datos_test["perfil_consumo"].copy()
```

```

preprocess_rf = ColumnTransformer(
    transformers=[
        ("num", SimpleImputer(strategy="median"), suple_numericas),
        ("cat", Pipeline(steps=[
            ("imputer", SimpleImputer(strategy="most_frequent")),
            ("onehot", OneHotEncoder(handle_unknown="ignore"))
        ]), suple_categoricas)
    ],
    remainder="drop"
)

```

```

In [24]: cv = StratifiedKFold(
    n_splits=5,
    shuffle=True,
    random_state=111104
)

rf = RandomForestClassifier(
    criterion="entropy",
    bootstrap=True,
    class_weight="balanced_subsample",
    random_state=111104,
    n_jobs=-1
)

modelo_rf = Pipeline(steps=[
    ("preprocess", preprocess_rf),
    ("rf", rf)
])

param_grid_rf = {
    "rf__n_estimators": [300, 500],
    "rf__max_samples": [0.7, 0.8, 1.0],
    "rf__max_features": ["sqrt"],
    "rf__max_depth": [10, 16, None],
    "rf__min_samples_leaf": [10, 25],
    "rf__min_samples_split": [145, 290],
}

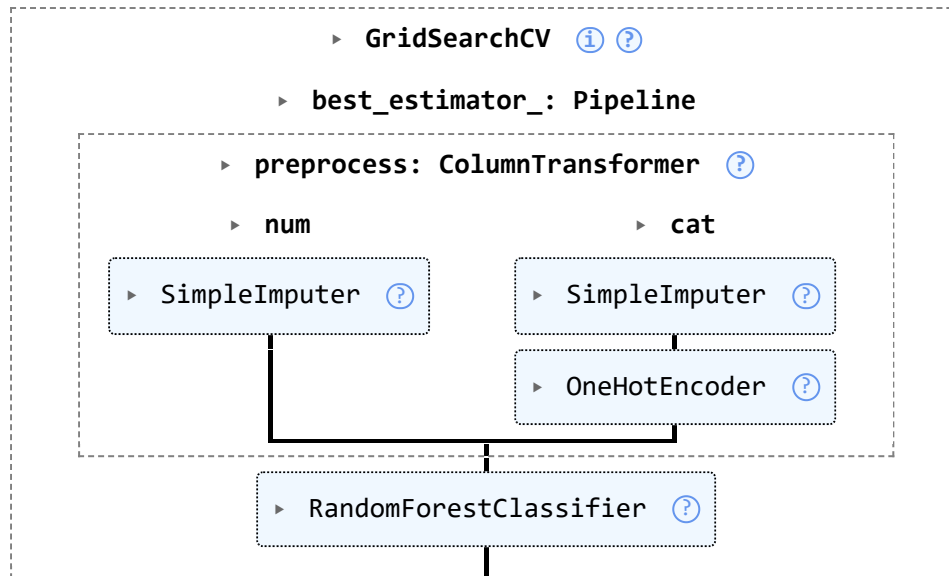
grid_rf = GridSearchCV(
    estimator=modelo_rf,
    param_grid=param_grid_rf,
    scoring={
        "accuracy": "accuracy",
        "balanced_accuracy": "balanced_accuracy",
        "f1_macro": "f1_macro"
    },
    refit="balanced_accuracy",
    cv=cv,
    n_jobs=1,
    verbose=1,
    return_train_score=True
)

grid_rf.fit(X_train_rf, y_train_rf)

```

Fitting 5 folds for each of 72 candidates, totalling 360 fits

Out[24]:



In [25]:

```
resultados_cv_rf = pd.DataFrame(grid_rf.cv_results_).copy()

resultados_cv_rf["n_estimators"] = resultados_cv_rf["param_rf__n_estimators"].as
resultados_cv_rf["max_samples"] = resultados_cv_rf["param_rf__max_samples"].as
resultados_cv_rf["max_features"] = resultados_cv_rf["param_rf__max_features"].as
resultados_cv_rf["max_depth"] = resultados_cv_rf["param_rf__max_depth"]
resultados_cv_rf["min_samples_leaf"] = resultados_cv_rf["param_rf__min_samples_l
resultados_cv_rf["min_samples_split"] = resultados_cv_rf["param_rf__min_samples_

cols_rf = [
    "n_estimators",
    "max_samples",
    "max_features",
    "max_depth",
    "min_samples_split",
    "min_samples_leaf",
    "mean_test_accuracy",
    "std_test_accuracy",
    "mean_test_balanced_accuracy",
    "std_test_balanced_accuracy",
    "mean_test_f1_macro",
    "std_test_f1_macro",
    "mean_train_balanced_accuracy"
]

resultados_rf_ordenados = resultados_cv_rf[cols_rf].sort_values(
    "mean_test_balanced_accuracy",
    ascending=False
)

display(resultados_rf_ordenados.head(15).round(4))
```


	n_estimators	max_samples	max_features	max_depth	min_samples_split	min_sample
1	500	0.7	sqrt	10	145	
40	300	1.0	sqrt	16	145	
64	300	1.0	sqrt	None	145	
32	300	0.8	sqrt	16	145	
16	300	1.0	sqrt	10	145	
57	500	0.8	sqrt	None	145	
41	500	1.0	sqrt	16	145	
56	300	0.8	sqrt	None	145	
68	300	1.0	sqrt	None	145	
69	500	1.0	sqrt	None	145	
49	500	0.7	sqrt	None	145	
33	500	0.8	sqrt	16	145	
45	500	1.0	sqrt	16	145	
17	500	1.0	sqrt	10	145	
0	300	0.7	sqrt	10	145	

```
In [26]: top_rf = resultados_cv_rf.sort_values(
        by="mean_test_balanced_accuracy",
        ascending=False
    ).head(5).copy()

    top_rf["etiqueta_eje_y"] = top_rf.apply(
        lambda r: (
            f"B={int(r['n_estimators'])}, "
            f"sample={r['max_samples']:.1f}, "
            f"mtry={r['max_features']}, "
            f"depth={r['max_depth']}, "
            f"leaf={int(r['min_samples_leaf'])}, "
            f"split={int(r['min_samples_split'])}"
        ),
        axis=1
    )

    columnas_splits_rf = [f"split{i}_test_balanced_accuracy" for i in range(5)]

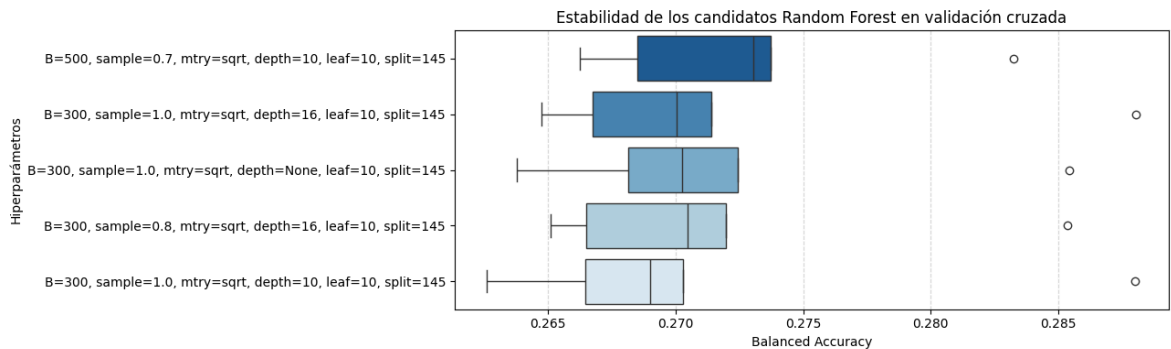
    df_boxplot_rf = top_rf.melt(
        id_vars=["etiqueta_eje_y"],
        value_vars=columnas_splits_rf,
        var_name="Fold",
        value_name="Balanced Accuracy"
    )

    plt.figure(figsize=(13, 4))
    sns.boxplot(
        data=df_boxplot_rf,
```

```

    x="Balanced Accuracy",
    y="etiqueta_eje_y",
    palette="Blues_r",
    orient="h"
)
plt.title("Estabilidad de los candidatos Random Forest en validación cruzada")
plt.xlabel("Balanced Accuracy")
plt.ylabel("Hiperparámetros")
plt.grid(axis="x", linestyle="--", alpha=0.5)
plt.tight_layout()
plt.show()

```



```

In [27]: mejor_rf = Pipeline(steps=[
    ("preprocess", preprocess_rf),
    ("rf", RandomForestClassifier(
        n_estimators=500,
        max_samples=0.7,
        max_features="sqrt",
        max_depth=10,
        min_samples_split=145,
        min_samples_leaf=10,
        criterion="entropy",
        bootstrap=True,
        class_weight="balanced_subsample",
        random_state=111104,
        n_jobs=-1
    ))
])

mejor_rf.fit(X_train_rf, y_train_rf)

def evaluar_clasificador(nombre_modelo, muestra, modelo, X, y):
    y_pred = modelo.predict(X)

    return {
        "modelo": nombre_modelo,
        "muestra": muestra,
        "accuracy": accuracy_score(y, y_pred),
        "balanced_accuracy": balanced_accuracy_score(y, y_pred),
        "f1_macro": f1_score(y, y_pred, average="macro"),
        "kappa": cohen_kappa_score(y, y_pred)
    }

metricas_rf = pd.DataFrame([
    evaluar_clasificador(
        "Random Forest",
        "train",
        mejor_rf,
        X_train_rf,

```

```

        y_train_rf
    ),
    evaluar_clasificador(
        "Random Forest",
        "test",
        mejor_rf,
        X_test_rf,
        y_test_rf
    )
])

display(metricas_rf.round(4))

```

	modelo	muestra	accuracy	balanced_accuracy	f1_macro	kappa
0	Random Forest	train	0.2807	0.3259	0.2469	0.1935
1	Random Forest	test	0.2447	0.2555	0.1992	0.1482

```

In [28]: y_pred_rf = mejor_rf.predict(X_test_rf)

print(classification_report(
    y_test_rf,
    y_pred_rf,
    zero_division=0
))

labels_rf = mejor_rf.named_steps["rf"].classes_

matriz_rf_norm = confusion_matrix(
    y_test_rf,
    y_pred_rf,
    labels=labels_rf,
    normalize="true"
)

soportes = y_test_rf.value_counts().to_dict()

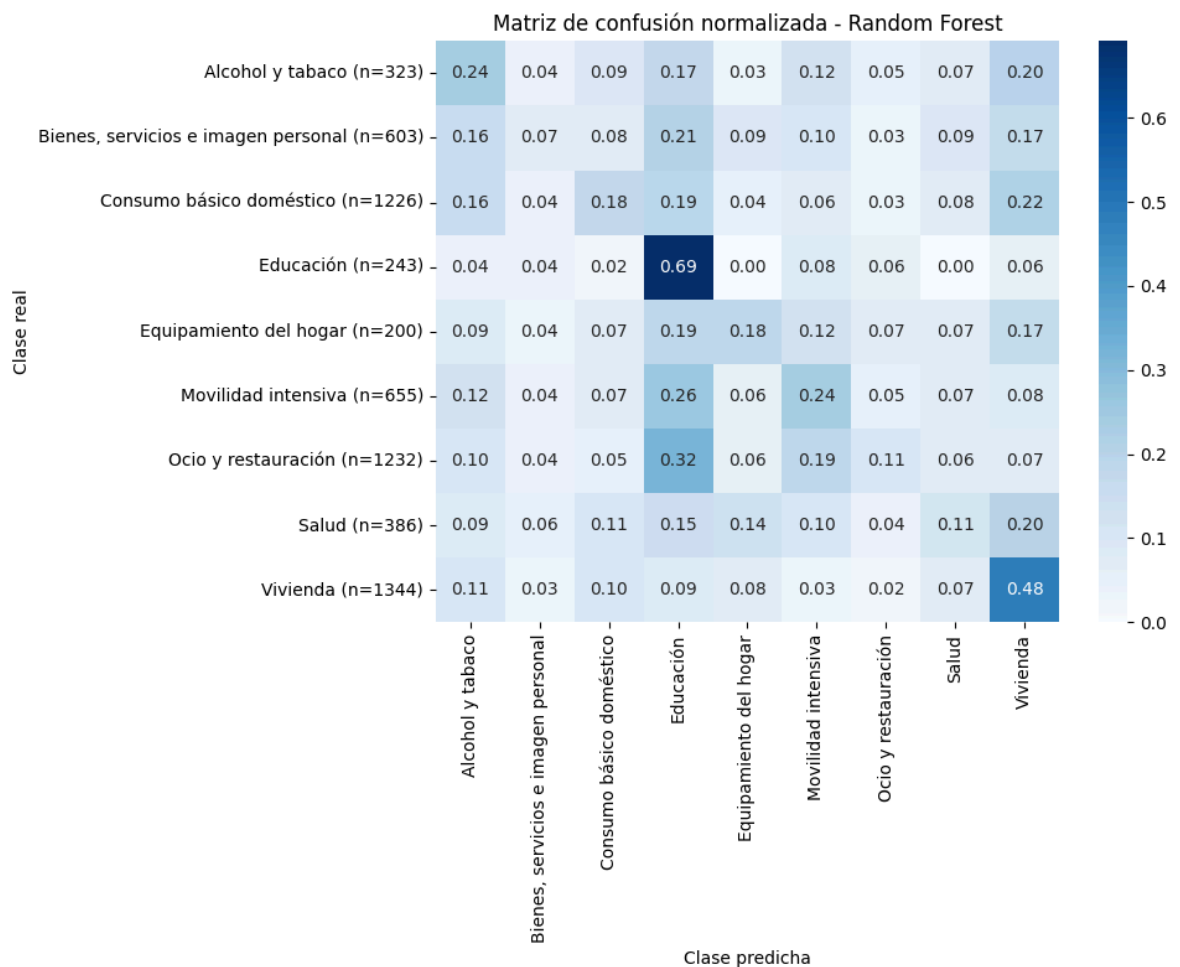
labels_con_n = [
    f"{label} (n={soportes[label]})"
    for label in labels_rf
]

plt.figure(figsize=(10, 8))
sns.heatmap(
    matriz_rf_norm,
    annot=True,
    fmt=".2f",
    cmap="Blues",
    xticklabels=labels_rf,
    yticklabels=labels_con_n
)

plt.title("Matriz de confusión normalizada - Random Forest")
plt.xlabel("Clase predicha")
plt.ylabel("Clase real")
plt.title("Matriz de confusión normalizada - Random Forest")
plt.tight_layout()
plt.show()

```

	precision	recall	f1-score	support
Alcohol y tabaco	0.10	0.24	0.14	323
Bienes, servicios e imagen personal	0.15	0.07	0.09	603
Consumo básico doméstico	0.36	0.18	0.24	1226
Educación	0.12	0.69	0.21	243
Equipamiento del hogar	0.09	0.18	0.12	200
Movilidad intensiva	0.23	0.24	0.23	655
Ocio y restauración	0.44	0.11	0.18	1232
Salud	0.10	0.11	0.11	386
Vivienda	0.48	0.48	0.48	1344
accuracy			0.24	6212
macro avg	0.23	0.26	0.20	6212
weighted avg	0.32	0.24	0.25	6212



```
In [29]: rf_final = mejor_rf.named_steps["rf"]

feature_names = mejor_rf.named_steps["preprocess"].get_feature_names_out()

importancias_rf = pd.DataFrame({
    "variable": feature_names,
    "importancia": rf_final.feature_importances_
}).sort_values("importancia", ascending=False)

display(importancias_rf.head(20).round(4))
```

	variable	importancia
4	num__log_gasto_total_crudo	0.2219
0	num__tamano_ocde	0.0868
3	num__n_ocupados	0.0687
32	cat__estudios_Alto	0.0433
31	cat__ocupado_1	0.0406
46	cat__edad_Más de 65	0.0393
39	cat__tipo_hogar_Pareja con hijos	0.0386
30	cat__ocupado_0	0.0380
1	num__n_menores	0.0327
2	num__n_mayores	0.0308
34	cat__estudios_Medio	0.0260
40	cat__tipo_hogar_Pareja sin hijos	0.0226
45	cat__edad_46-65	0.0219
33	cat__estudios_Bajo	0.0172
42	cat__tipo_hogar_Persona sola ≥ 65	0.0167
28	cat__hombre_0	0.0164
29	cat__hombre_1	0.0151
26	cat__tam_municipio_Muy Grande	0.0150
27	cat__tam_municipio_Pequeño	0.0136
44	cat__edad_31-45	0.0134

Selección e interpretación del mejor modelo

```
In [30]: tabla_comparativa = pd.concat(
    [metricas_logit_ridge, metricas_arbol, metricas_bagging, metricas_rf],
    ignore_index=True
)

cols = ["modelo", "muestra", "accuracy", "balanced_accuracy", "f1_macro", "kappa"]
tabla_comparativa = tabla_comparativa.loc[:, cols]

display(tabla_comparativa.round(4))
```

	modelo	muestra	accuracy	balanced_accuracy	f1_macro	kappa
0	Logística multinomial Ridge	train	0.2674	0.2915	0.2290	0.1772
1	Logística multinomial Ridge	test	0.2601	0.2579	0.2128	0.1649
2	Árbol de clasificación	train	0.2412	0.2867	0.2090	0.1555
3	Árbol de clasificación	test	0.2165	0.2379	0.1805	0.1269
4	Bagging	train	0.2676	0.2971	0.2264	0.1755
5	Bagging	test	0.2413	0.2431	0.1937	0.1428
6	Random Forest	train	0.2807	0.3259	0.2469	0.1935
7	Random Forest	test	0.2447	0.2555	0.1992	0.1482

```
In [31]: df_test = tabla_comparativa[tabla_comparativa['muestra'] == 'test'].copy()

df_melted = df_test.melt(
    id_vars='modelo',
    value_vars=['accuracy', 'balanced_accuracy', 'f1_macro', 'kappa'],
    var_name='Métrica',
    value_name='Valor'
)

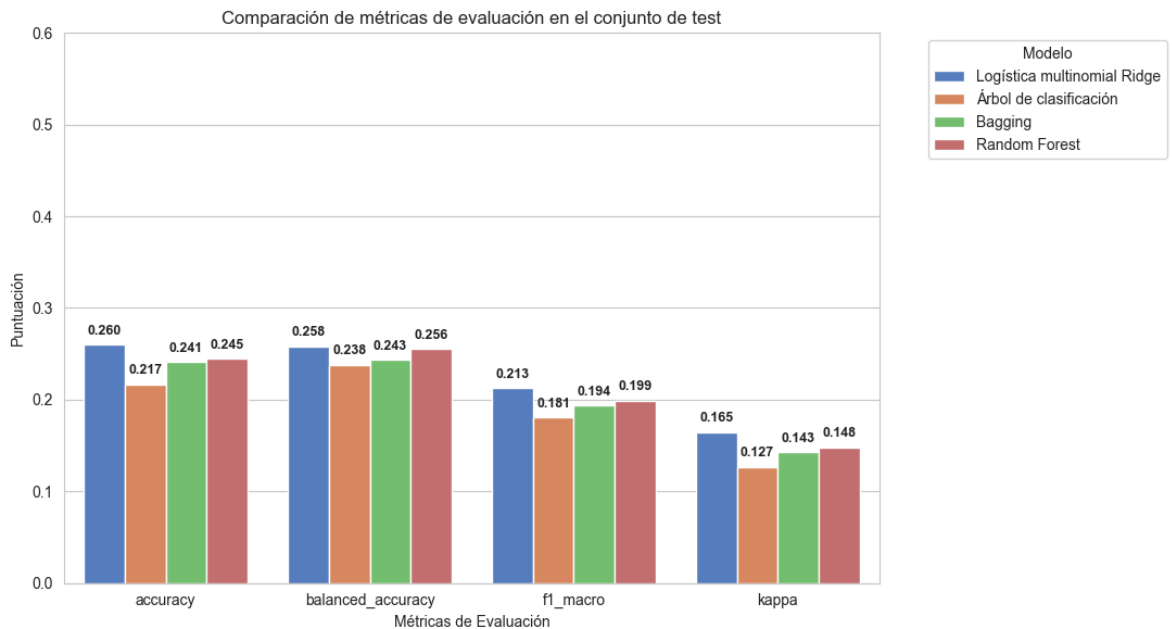
plt.figure(figsize=(11, 6))
sns.set_style("whitegrid")

ax = sns.barplot(
    data=df_melted,
    x='Métrica',
    y='Valor',
    hue='modelo',
    palette='muted'
)

plt.ylim(0, 0.6)
plt.title('Comparación de métricas de evaluación en el conjunto de test')
plt.ylabel('Puntuación')
plt.xlabel('Métricas de Evaluación')
plt.legend(title='Modelo', bbox_to_anchor=(1.05, 1), loc='upper left')

for p in ax.patches:
    if p.get_height() > 0:
        ax.annotate(format(p.get_height(), '.3f'),
                    (p.get_x() + p.get_width() / 2., p.get_height()),
                    ha = 'center', va = 'center',
                    xytext = (0, 9),
                    textcoords = 'offset points',
                    fontsize=9,
                    fontweight='bold')

plt.tight_layout()
plt.show()
```



```
In [32]: reporte_logit_ridge = classification_report(
    y_test_final,
    y_pred_test_ridge,
    output_dict=True,
    zero_division=0
)

tabla_clases_logit_ridge = (
    pd.DataFrame(reporte_logit_ridge)
    .T
    .loc[labels_ridge, ["precision", "recall", "f1-score", "support"]]
    .rename(columns={
        "precision": "precision",
        "recall": "sensibilidad",
        "f1-score": "f1_score",
        "support": "n"
    })
)

display(tabla_clases_logit_ridge.round(4))
```

	precision	sensibilidad	f1_score	n
Alcohol y tabaco	0.1033	0.2353	0.1435	323.0
Bienes, servicios e imagen personal	0.1794	0.0896	0.1195	603.0
Consumo básico doméstico	0.3765	0.1566	0.2212	1226.0
Educación	0.1171	0.5432	0.1927	243.0
Equipamiento del hogar	0.0686	0.2000	0.1022	200.0
Movilidad intensiva	0.2652	0.2870	0.2757	655.0
Ocio y restauración	0.4142	0.1372	0.2061	1232.0
Salud	0.1143	0.1451	0.1279	386.0
Vivienda	0.5260	0.5275	0.5267	1344.0

```

In [33]: clasificador_logit = modelo_logit_ridge_final.named_steps["logit"]
coeficientes_brutos = clasificador_logit.coef_

nombres_variables = X_train_final.columns
clases_perfiles = modelo_logit_ridge_final.classes_

df_coef_bruto = pd.DataFrame(coeficientes_brutos, index=clases_perfiles, columns=
nombres_variables)

categoria_referencia = 'Vivienda'

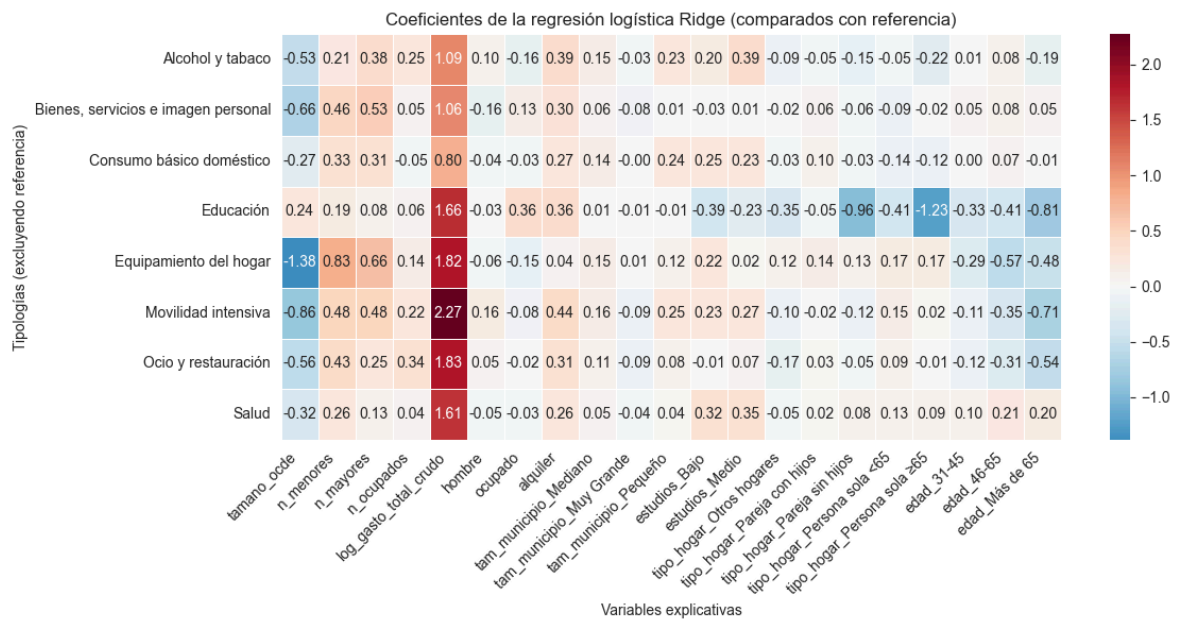
df_coef_j1 = df_coef_bruto.sub(df_coef_bruto.loc[categoria_referencia], axis=1)

df_coef_j1 = df_coef_j1.drop(categoria_referencia)

variables_clave = [c for c in nombres_variables if 'c_autonoma' not in c]

plt.figure(figsize=(12, 6))
plt.title("Coeficientes de la regresión logística Ridge (comparados con referencia)")
sns.heatmap(df_coef_j1[variables_clave], cmap="RdBu_r", center=0, annot=True, fr
plt.xlabel("Variables explicativas")
plt.ylabel("Tipologías (excluyendo referencia)")
plt.xticks(rotation=45, ha="right")
plt.tight_layout()
plt.show()

```



```

In [34]: y_score = modelo_logit_ridge_final.predict_proba(X_test_final)
y_test_bin = label_binarize(y_test_final, classes=clases_perfiles)
n_classes = len(clases_perfiles)

fpr = dict()
tpr = dict()
roc_auc = dict()
auc_resumen = []

for i in range(n_classes):
    fpr[i], tpr[i], _ = roc_curve(y_test_bin[:, i], y_score[:, i])
    roc_auc[i] = auc(fpr[i], tpr[i])
    auc_resumen.append({
        "Perfil de Consumo (Categoría Respuesta)": clases_perfiles[i],
        "Valor Estimado (Test)": round(roc_auc[i], 4)
    })

```



```

    })

tabla_auc_definitiva = pd.DataFrame(auc_resumen)
display(tabla_auc_definitiva)

plt.figure(figsize=(10, 8))

colores = sns.color_palette("husl", n_classes)
for i, color in zip(range(n_classes), colores):
    plt.plot(fpr[i], tpr[i], color=color, lw=2, label=f'{clases_perfiles[i]} (AU

plt.plot([0, 1], [0, 1], 'k--', linewidth=2)
plt.xlim([0.0, 1.0])
plt.ylim([0.0, 1.05])
plt.title('Curvas ROC para Regresión Logística Ridge')
plt.xlabel('1 - Especificidad')
plt.ylabel('Sensibilidad')
plt.legend(loc="lower right", fontsize=9)
plt.tight_layout()
plt.show()

```

	Perfil de Consumo (Categoría Respuesta)	Valor Estimado (Test)
0	Alcohol y tabaco	0.6458
1	Bienes, servicios e imagen personal	0.5979
2	Consumo básico doméstico	0.6766
3	Educación	0.8153
4	Equipamiento del hogar	0.6313
5	Movilidad intensiva	0.7324
6	Ocio y restauración	0.7369
7	Salud	0.6461
8	Vivienda	0.8079

